



University of St. Gallen

School of Management, Economics,
Law, Social Sciences, International Affairs and Computer Science

Group Project:
Study Planner – Streamlit Web Application
Documentation

Submitted by:

Xavier Wanner (20-410-908)

Tim Silver Blum (22-918-791)

Justus Baum (22-612-956)

Niklas Knotzer (22-606-719)

Professor:

Mario Silic

4,799 Skills: Programming – Introduction Level

8,799 Skills: Programming with Advanced Computer Languages

Date of Submission:

15/06/2026

Project overview and aim

This document provides a commentary on the design and implementation of a Streamlit-based Study Planner application, a web-based application built with the Streamlit framework in Python. The goal of the program is to allow students to schedule and manage their exam preparation by adding subjects, study time needed, monitoring their progress, and visualising a rolling weekly schedule.

The application therefore combines planning, progress tracking, and risk awareness in one simple interface. The application is intended as a practical academic tool: lightweight enough to run locally without any configuration, yet complete enough to support a student managing several subjects with different exam dates and workloads.

Technical Overview

The application is written entirely in Python and relies on two dependencies: the Streamlit library and the standard datetime module. No database, external API, or additional package is required. Therefore, the application can be launched locally with a single terminal command after installing Streamlit via pip.

The code is structured around pure functions: each function has a single, clearly defined responsibility and does not modify any state outside of the `session_state` dictionary. Calculation functions such as progress computation, workload classification, and priority assignment are kept separate from the display functions that render each page. This separation makes the logic easier to test and adjust independently of the interface. Data is stored as a list of Python dictionaries, one per subject, which provides a straightforward and readable structure while avoiding the unnecessary overhead of defining custom classes for a relatively small application.

Unlike a standard Python script, this application cannot be run by executing the file directly in an editor such as VS Code or by calling it with the Python interpreter alone. It must be launched through the Streamlit framework using the command `streamlit run filename.py` in the terminal, which starts a local web server and opens the interface automatically in the default browser. Streamlit must therefore be installed beforehand.

General Architecture

The application is organised around five pages, each accessible from a sidebar navigation panel. All subject data is stored in Streamlit's session state as a list of dictionaries, allowing information to persist across page transitions within a single session. A central dispatcher function reads the sidebar selection and calls the appropriate page function. Each page is fully self-contained.

Dashboard

The Dashboard is the default landing page and is the principal overview of the student's preparation. It displays four summary metrics shown side by side: the total number of subjects, total required study hours, remaining hours to study, and overall completion as a percentage. A progress bar under these figures provides an immediate visual sense of how far along the student is across all subjects combined.

Below the summary, the individual advancement per subject is shown. The card shows the subject name alongside its priority level, colour-coded in green for low, orange for medium, and red for high. It also shows a subject-level progress bar, the number of days remaining until the exam, and the recommended daily study hours with the corresponding workload label. At the bottom of the page, a risk analysis message either lists “high risk” if there are some exams in the next three days or more than 5 hours needed in one day to study a specific topic. Otherwise, it confirms that no critical workload has been detected.

Add Subject

This page presents a form where the student enters a subject name, the number of days until the exam, and the total number of study hours required. On submission, the application validates the input and,

if all checks pass, appends the new subject to the session state. The page is the main entry point for populating the planner and is where the most important constraints are enforced, as described in the section explaining the rationale behind the constraints. If the total study time needed exceeds 14 hours per day, the new input is not added at all to the lists of subjects.

Update Progress

The Update Progress page allows the student to register hours that have already been studied for a selected subject. A dropdown lists all existing subjects, and a number input accepts the additional hours to record. The completed hours are incremented and automatically capped at the subject's total, so the progress value never exceeds 100 percent even if the student enters more hours than originally scheduled. The page reruns immediately after saving so that the dashboard reflects the new state on the next visit.

Weekly Study Plan

The Weekly Plan page generates a rolling 7-day schedule starting from the current day. For each subject that still must be studied, the remaining workload is distributed evenly across the lower of seven days or the number of days left until the exam. Seven columns are shown, one per day, with each column listing the subjects to study and the hours allocated per topic. Days with no scheduled study are labelled as free. As the schedule is anchored to today rather than to a fixed Monday-to-Sunday grid, it remains accurate regardless of when the student opens the application.

Delete Subject

The Delete Subject page allows the student to remove a subject from the planner. A dropdown lists all current subjects and a clearly labelled button confirms the deletion. The subject is filtered out of the session state immediately, and the page reruns to keep the interface consistent. This page is intentionally simple, as its purpose is to keep the subject list clean when a subject is no longer relevant.

Explanation of Constraints

Several constraints were added to the application to ensure that the study plans are realistic, to enable students to use them in real life.

Maximum Daily Study Load of 14 Hours

When a new subject is added, the application first checks whether the combined daily study demand across all subjects, including the one being added, would exceed 14 hours. This is computed by summing, for each existing subject, the remaining hours divided by the days left, then adding the projected daily demand of the new subject. If the total exceeds 14 hours, the subject is rejected and an error message is displayed. The threshold of 14 hours was chosen as an upper bound on what a student could realistically dedicate to studying in a single day while being productive. This prevents the planner from generating schedules that look complete on paper but are impossible to follow in practice for students.

Risk Detection

A subject is defined as high-risk if one of two conditions is met: the required daily study hours exceed five, or fewer than three days remain before the exam. These thresholds classify situations where the student is either running severely short on time or facing an unsustainable daily workload. High-risk subjects are highlighted in the dashboard cards with a red priority label and are also listed explicitly in the risk warning at the bottom of the dashboard, to catch the student's attention.

Input Validation

All forms in the application enforce strict input validation. Subject names must be non-empty after whitespace is removed, and duplicates are rejected through a case-insensitive comparison against all existing subject names. Numeric inputs for days and hours are constrained to positive values by the form widgets themselves, and the update progress form additionally rejects a zero entry to prevent accidental submissions. These checks are necessary because the calculation functions divide by the

number of days remaining and by total hours: invalid inputs would produce division errors or meaningless results that would propagate silently through the rest of the application.

Progress Capping

When a student inputs completed hours, the application caps the total at the subject's required hours. This means that studying beyond the original estimate does not push progress above 100 percent, which would distort the overall summary metrics on the dashboard. The same cap is applied in the progress calculation function itself, providing a second layer of protection regardless of how the data is modified.

Limitations and possible improvements

Despite its functional design, the application carries multiple limitations that make future improvements possible. Four limitations will be mentioned in what follows. First, all data is stored exclusively in Streamlit's session state, meaning every subject and progress entry is permanently lost upon closing or refreshing the browser. Second, the weekly plan distributes workload evenly across available days, with no option to block out unavailable days such as weekends, personal commitments, or focusing on only one exam for a specific day. Third, the application lacks any notification system to alert the student when an exam is approaching or a subject becomes high-risk outside of the app. Fourth, exam deadlines are entered as a number of days remaining rather than an actual date, which is less intuitive and more error-prone.

Possible improvements could be the following. Replacing the days-remaining input with the actual exam date would make the planner more natural to use. Adding a custom availability selector would allow the weekly schedule to reflect the student's actual free days rather than assuming full availability. An export feature generating a PDF or CSV of the study plan would make it easy to print or share. Another possible improvement would be adding persistent data storage, for example through a local CSV file or a lightweight SQLite database. This would allow students to close the application and continue working with the same data later.